Chapter 3: Investigating the Memory of Events Using N-Grams, Controlled Vocabulary, and Joins

Computational methods support inquiry into how the discourses and events in the past shape current-day political change. It is common in speeches for political actors to reference the events of the past – whether American politicians reference the Boston Tea Party or British parliamentarians reference the events of the Second World War. Speakers in parliament also acknowledge contemporary public protests as a source of political legitimacy. But the number, intensity, and specific events referenced change over time. What can we learn about how political reforms were made in the past from reviewing how politicians of the past referenced events?

In this chapter, we will explore questions such as these: In the lead-up to the Great Reform Act of 1832, when the middle class got the vote in Britain, which historical references were on the table? Which members of parliament contributed the most to the debate about the Abolition of Slavery in 1833, and of these speakers, whose language reflected the most on the lived experience of toil and cruelty in the system of slavery? Which historical and contemporary events were referenced the most by members of parliament in the era of the Second Reform Act of 1867, when working-class people got the vote for the first time?

This chapter builds upon a sense of how to leverage computational historical thinking to create insight about history. It introduces counting n-grams, or multiword phrases. It also introduces the use of a "controlled vocabulary," that is, a historian-curated list of important words and phrases, to explore which events were spoken about in parliament during the 1830s.

This chapter builds on the previous to introduce analysts to fundamental data processing techniques in R, commonly used for preparing data for analysis. A key task in data analysis is summarizing a dataset's characteristics to gain insights into the features of the corpus. Scholars have drawn attention to how th immediacy of computational text analysis–its perceived automation in transforming a raw, messy corpus into a curated object of analysis–masks the degree of transformation that is enacted on the text's "original" and intermediary data structures (Palermo 2017). In this chapter, we draw attention to each step in data processing, emphasizing how intermediary data wrangling steps contributes to the creation of new knowledge.

One essential concept in data processing is "joining" datasets. A join combines two datasets into one, allowing us to analyze parliamentary texts alongside metadata such as speaker information, dates, and significant events. Additionally, this chapter teaches readers how to work with event-based data, including years, months, and dates. By moving through the data processing steps one-by-one, we draw on Paul Fyfe's framing of "data mining" as "data archaeology," shifting attention away from analysis as a final product and instead foregrounding the transformational work of data processing as a central part of the analytical process (Fyfe 2016).

As with the previous chapter, this chapter will emphasize the skills of moving from analyzing text as data to performing a close reading of historical speeches in context. It models the iterative process of working with a controlled vocabulary and reading words in context with the idea of demonstrating a method of inquiry about how and why events happen. The approach we model provides analysts with the tools of deciding for themselves how the patterns identified throughout text mining relate back to their primary sources, that is,

the full Hansard debates. As we had previously discussed, our analytic process is iterative, emphasizing that researchers transition from data processing to counting words to applying metrics on those word counts to visualizing trends, and then repeating this deductive process. Throughout the process, it may prove valuable to inspect and re-evaluate the intermediary steps that were taken to arrive at any conclusions about the corpus. This cyclical process may lead to additional historical questions and associations, which may mean more measuring, more visualization, and more guided reading in pursuit of a historical analysis.

```
# Load the packages used in this chapter
library(hansardr)   # Access the Hansard data
library(tidyverse)  # Load the tidyverse packages
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.0     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.1     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.1
## v purrr     1.2.1
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidytext)   # Tokenize and analyze text
library(lubridate)  # Work with dates
library(viridis)    # Color palettes for visualizations
```

```
## Loading required package: viridisLite
```

Joining Debate Metadata

We have previously seen how to load the text from a single decade of the Hansard debates. But what should we do if we want to work with not only the text, but also important information like the name of the speaker who gave the speech and the date on which the speech was given?

First we can load the text data from the 1860s speeches, recalling that one of the columns, `sentence_id`, refers to the number of each sentence and its location in the nineteenth-century printed volumes of Hansard's parliamentary debates, referenced by series ("S"), volume ("V"), and page ("P"). Sentences are listed in the order in which they appeared on the page.

```
# Load the 1860s Hansard dataset
# - data() loads the dataset into the R environment
data("hansard_1860")
```

```
# Display the sample sentences as a formatted table
# - kable() formats the data as a table
# - column_spec() widens the text column for readability
```

```r
df_head %>%
  kable(caption = "Sample sentences from 1860s Hansard, including each sentence's
                   unique identifier and the corresponding text as extracted from
                   the debate transcripts.") %>% # Format the results as a table
  column_spec(2, width = "12cm") # Set the width of the text column
```

Table 3.1: Sample sentences from 1860s Hansard, including each sentence's unique identifier and the corresponding text as extracted from the debate transcripts.

sentence_id	text
S3V0156P0_0	which had been prorogued successively to the 27th October, thence to the 15th December, and thence to the 24th January, met this day for Despatch of Business.
S3V0156P0_1	being seated on the Throne, and the Commons being at the Bar, with their Speaker, HER MAJESTY was pleased to make a most gracious Speech to both Houses of Parliament, as follows:—
S3V0156P0_2	in rising to move that an humble Address be presented by this House in answer to Her Majesty's most gracious Speech, said:—
S3V0156P0_3	My Lords, it is with diffidence that I rise to address your Lordships for the first time, and the greater is that diffidence when I recollect that although for many years I had the honour of a seat in the other House of Parliament, I remained throughout that time a silent actor in all the scenes to which my duties called me; but I will be brief, and the observations which I make shall be few, and I trust you will not weigh in too nice a balance any expressions which may fall from me on this occasion.
S3V0156P0_4	We hear from Her gracious Majesty that it is with gratification

Next, we can load the metadata corresponding with the Hansard debate text.

```r
# Load the 1860s debate metadata
data("debate_metadata_1860")
```

Table 3.2: Debate-level metadata from 1860s Hansard, including each sentence's identifier, the date of the speech, and the title of the corresponding debate.

sentence_id	speechdate	debate
S3V0156P0_0	1860-01-24	MEETING OF THE PARLIAMENT.
S3V0156P0_1	1860-01-24	THE QUEEN'S SPEECH.
S3V0156P0_2	1860-01-24	ADDRESS IN ANSWER TO HER MAJESTY'S SPEECH.
S3V0156P0_3	1860-01-24	ADDRESS IN ANSWER TO HER MAJESTY'S SPEECH.
S3V0156P0_4	1860-01-24	ADDRESS IN ANSWER TO HER MAJESTY'S SPEECH.

`hansardr` includes detailed contextual information for each of the speeches, using the same system of references that we saw in the previous data set – series, volume, page, and sentence number. In the `debate_metadata` dataset, however, there is no "text" column, but rather a series of columns that illuminate multiple "fields" of each sentence: the speech date (recorded as "speechdate" within the data) for when the speech was given and the official title of the "debate" in which the sentence was spoken.

In addition to the debate metadata, `hansardr` provides two additional metadata files: `speaker_metadata_1860` and `file_metadata_1860`.

```
# Load 1860s Hansard speaker metadata
data("speaker_metadata_1860")
```

Table 3.3: Speaker metadata for sample sentences from the 1860 Hansard corpus, including the recorded speaker name, the algorithmically suggested speaker match, and indicators for name ambiguity, fuzzy matching, and ignored cases.

sentence_id	speaker	suggested_speaker
S3V0156P0_0	THE PARLIAMENT,	NA
S3V0156P0_1	THE QUEEN	peter_quinn_4670
S3V0156P0_2	EARL FITZWILLIAM,	charles_fitzwilliam_1441
S3V0156P0_3	EARL FITZWILLIAM,	charles_fitzwilliam_1441
S3V0156P0_4	EARL FITZWILLIAM,	charles_fitzwilliam_1441

The `speaker_metadata_1860` dataset provides information about the "speaker" of each sentence as the MP's name was recorded in the original Hansard debates. It also includes a "suggested speaker" column, which contains a disambiguated version of the speaker names where ambiguous or inconsistent references to the same individual have been resolved into a single, standardized name, allowing for more accurate identification of MPs across the corpus.

The "suggested speakers" came from a large-scale disambiguation effort where the authors of this book disambiguated speaker names, building upon the work completed by the Digging into Linked Parliamentary Data Project and the Egger and Spirling database (Eggers and Spirling 2014; Winters and Steer 2013). Our disambiguation effort was designed to address these problems: speakers may have been called only by temporary office titles (like "Prime Minister"); speakers may have been called only by their peerage titles (such as "Duke" or "Viscount");a single name might have several permutations (such as W. Gladstone, W. E. Gladstone, or William Gladstone all referring to the same Mr. William Ewart Gladstone); speakers may have shared the exact name with someone else who was active during the same period (like the two different Sir Robert Peels in the early 19th-century); speaker names may contain OCR errors; speaker names might have spelling inconsistencies due to how they were originally recorded during the 19th-century; some speakers were misrecorded, such as they were called by the wrong title.

By providing both the original speaker field as well as the disambiguated version, our version of the Hansard dataset preserves the original historical recording and draws attention to the data transformation that occurred in the speaker field before the corpus was packed into `hansardr`. The purpose of including the "suggested speakers" field is to support computational analysis by linking variant or ambiguous names to consistent, canonical speaker identities for a more reliable analysis.

However, we also advise discretion. While disambiguation significantly improves analysis by standardizing speaker identities, it is not without limitations. In some cases, speakers may be incorrectly matched or mislabeled by the algorithm. In others, no suggested speaker name is provided because the necessary metadata was unavailable, and the sheer number of speakers made manual correction infeasible within our project timeline. To address these challenges, we have uploaded the Hansard corpus to Democracy Viewer, an online text mining and data sharing platform where researchers can also submit corrections to the corpus (Buongiorno et al., 2023). Over time, we plan to incorporate data set improvements into future releases of the `hansardr` package.

Additionally, disambiguation can obscure the nuances of the original records, such as shifts in titles or the historical importance of misspellings for understanding contemporary spelling conventions or clerical practices. Due to the historical richness of original speaker names, we provide both the original speaker field and the disambiguated version, so that researchers can critically assess and, if desired, use the original speaker name data instead.

Lastly, `hansardr` provides file metadata. For example, the `file_metadata_1860` dataset provides metadata about the records themselves, including the ID for the original source file, and the column in which the text appears.

```
# Load 1860s Hansard file metadata
data("file_metadata_1860")
```

Table 3.4: Source file metadata for selected sentences from the 1860 Hansard corpus, including the identifiers for the sentence, speech, and debate, as well as the source file, source image, and original column location.

sentence_id	speech_id	debate_id	src_file_id	src_image	src_column
S3V0156P0_0	258488	30416	S3V0156P0	S3V0156P0I0033	1
S3V0156P0_1	258489	30417	S3V0156P0	S3V0156P0I0033	1
S3V0156P0_2	258490	30418	S3V0156P0	S3V0156P0I0036	7
S3V0156P0_3	258490	30418	S3V0156P0	S3V0156P0I0036	7
S3V0156P0_4	258490	30418	S3V0156P0	S3V0156P0I0036	7

We have chosen to store the debate text (`hansard_1860`) separately from the contextual information about each speech (for example, `debate_metadata_1860` or `speaker_metadata_1860`), as well as the information about each hard copy file containing the speeches (`file_metadata_1860`). We subsetted the data into individual files because the data for each decade is quite large. Processing such a large dataset can exceed a computer's resources, such as its memory. In other cases, we chose to exclude certain fields from processing if they were not immediately relevant to our analysis, as doing so reduces memory usage and improves overall processing speed so that we can focus on our research instead of waiting too long for any intermediary processing step to complete. Furthermore, many operations in this book – for instance, counting words per decade – require just a few datasets loaded at a time.

Many kinds of analyses will require working with multiple datasets from `hansardr` joined together. We will use the function `left_join()` to join the annotated information about speech context with each of the speech text. A left join returns every record from the data frame specified by the left-most argument, and all the matched records from the data frame on the right.

```
# Add the debate metadata to the Hansard text
# - left_join() automatically detects that both datasets share the sentence_id column
# - it uses sentence_id to match each sentence with its corresponding metadata
hansard_1860 <- left_join(hansard_1860, debate_metadata_1860)

# Display the first six rows of the merged dataset
# - head() keeps the first six rows
# - str_trunc() shortens long text for readability
# - kable() formats the results as a table
hansard_1860 %>%
  head() %>% # Keep the first six rows
  mutate(text = str_trunc(text, 120)) %>% # Shorten the text column
  kable(caption = "Text excerpts from the 1860 Hansard corpus alongside their associated
                   metadata, including the sentence identifier, speech date, and debate title.") # Form
```

Table 3.5: Text excerpts from the 1860 Hansard corpus alongside their associated metadata, including the sentence identifier, speech date, and debate title.

sentence_id	text	speechdate	debate
S3V0156P0_0	Which had been prorogued successively to the 27th October, thence to the 15th December, and thence to the 24th Januar...	1860-01-24	MEETING OF THE PARLIAMENT.
S3V0156P0_1	Being seated on the Throne, and the Commons being at the Bar, with their Speaker, HER MAJESTY was pleased to make a m...	1860-01-24	THE QUEEN'S SPEECH.
S3V0156P0_2	Rising to move that an humble Address be presented by this House in answer to Her Majesty's most gracious Speech, ...	1860-01-24	ADDRESS IN ANSWER TO HER MAJESTY'S SPEECH.
S3V0156P0_3	My Lords, it is with diffidence that I rise to address your Lordships for the first time, and the greater is that dif...	1860-01-24	ADDRESS IN ANSWER TO HER MAJESTY'S SPEECH.
S3V0156P0_4	We hear from Her gracious Majesty that it is with gratification	1860-01-24	ADDRESS IN ANSWER TO HER MAJESTY'S SPEECH.
S3V0156P0_5	She meets her Parliament, that She continues on terms of friendship with all foreign countries; but while there is th...	1860-01-24	ADDRESS IN ANSWER TO HER MAJESTY'S SPEECH.

Notice that the resulting dataset, `hansard_corpus_1860`, has information about speaker and date for each sentence of text.

Joins are a crucial concept for combining and analyzing data that is distributed across multiple datasets.

They allow us to link related information from different tables based on a shared key or common field thus enabling us to perform analyses of the records in relation to its metadata.

In the example above, both `hansard_1860` and `debate_metadata_1860` share a column called `sentence_id`. The shared `sentence_id` column acts as a common identifier, enabling us to link the speaker and date information from one dataset to the corresponding text of the speech in the other dataset. By using this common identifier, the two datasets can be "joined" together to create a single, unified dataset that we will use for text processing.

Thus joins are so important because it is through linking the records to their metadata that we are able to break text into subsets and analyze change over time, or create a biography of any individual speaker. We can explore the words invoked in parliament in the months leading up to August 15, 1867, when most working-class men in Britain finally achieved the right to vote.

Multiword Phrases

Beyond analyzing individual words, an analyst may want to explore multiword phrases. Multiword phrases contain formulations of concepts that had powerful political, social, and cultural meaning to speakers, for example, "the will of the people," a phrase that has been invoked by both radicals and conservatives to support extremely different imaginations of the people. That is, speakers who invoked "the will of the people" might have been bolstering the importance of their speech by gesturing towards their status as a representative of the workers whose labor is the source of all political legitimacy. On the other hand, some speakers might have used the same phrase – "the will of the people" – to kindle an imagination of parliament as a zone a traditional folk whose national identity legitimizes a politics of excluding from citizenship or rights people with certain racial or immigration status. The question becomes: which version of the people was being represented in the discourse in question?

The analyst cannot anticipate in advance how such a phrase might have been used. The only way of gaining knowledge from a count of phrases is to "validate" one theory or the other by gaining more information.

We could, in theory, contrive computational approaches to help us discern why people invoked "the will of the people." But in *Text Mining for Historical Analysis*, we often want to move from identifying a data-driven phenomenon to gaining more information about historical context by reading the actual speeches. Our approach modeled approach–moving from a visualization overview back to the data itself–provides insight into the rich nuances suggested by speakers by giving us perspective into how they invoke particular phrase in the original records. While visualizations might demonstrate that a parliamentarian invoked a phrase, they may not clearly demonstrate how that phrase was invoked.

Accordingly, in the sections below, we will begin by counting phrases, but that is not where we will end. We will return to the original speeches to read more deeply.

Finding Bigrams

While there are several approaches to finding multiword phrases, we will demonstrate finding them by tokenizing the text into bigrams.

The following code filters the speeches of the 1860s for the first months of 1867 before the vote on the Second Reform Act, which gave Britain's urban working-class men the right to vote. We also introduce a new package, `lubridate`, which can be installed in the usual way using `install.packages()`.

```
# Create a dataset containing speeches from 1867 before the Second Reform Act
# - mutate() extracts the year from the speech date
# - filter() keeps only speeches from 1867 through August 15
hansard_1867 <- hansard_1860 %>% # Start with the 1860s Hansard data
  mutate(year = year(speechdate)) %>% # Extract the year from the speech date
  filter(year == 1867, # Keep only speeches from 1867
         speechdate <= ymd("1867-08-15")) # Keep speeches on or before August 15, 1867
```

Now that we have filtered the dataset for our desired timeline, we can tokenize the text into bigrams. The order in which we perform data processing is important. Tokenizing after filtering ensures that we are working with a smaller and more focused dataset. Working with a smaller dataset ensures we do not exhaust our computer's resources or its processing capabilities. Skills for handling a large dataset on a smaller computer, like a laptop, is paramount to being able to answer questions while processing large corpora for historical analysis, as we describe in more detail in subsequent chapters.

The order of our steps is not arbitrary, but the outcome of a deliberate decision. We could have reversed them—for example, tokenizing the text before filtering by date. However, that approach would have used more of our computer's resources, as it would require processing a larger volume of irrelevant text before narrowing the records down to just the data needed for our inquiry into a given time period. With large datasets like Hansard, the sequence of processing steps can significantly impact whether the data remains manageable.

In turn, the limitations of any computing environment directly shape the types of analyses we prioritize. In other words, it is not only theoretical or interpretive concerns that determine our methodological choices, but also what is computationally feasible. Technical constraints become analytical constraints.

When working with large-scale historical data, researchers often adapt their questions, models, and scope to align with the practical boundaries of a computer's available memory, processing speed, and storage. Computational limitations might encourage researchers to think critically and strategically about what kinds of knowledge are worth pursuing, and how best to pursue them with the tools at hand.

Throughout *Text Mining for Historical Analysis* we have attempted to limit barriers to processing a dataset as large as Hansard through our data organization scheme that breaks each decade of text and related metadata files into their own files, and presents readers with an order of intermediary data processing that is more efficient for working with historical data in R.

The following code uses the `unnest_tokens()` function, which we previously used to split text into individual words. This time, we adjust the function to extract pairs of consecutive words instead. To focus on pairs of consecutive words instead, we set the token argument to n-grams, instructing R to create sequences of consecutive words, and the `n` argument to `2`, specifying that each sequence should consist of two words. The two arguments work together to extract bigrams.

```
# Generate bigrams (two-word sequences) from the text
# - unnest_tokens() tokenizes the text
# - token = "ngrams" creates sequences of words instead of single words
# - n = 2 creates two-word sequences (bigrams)
bigrams_1867 <- hansard_1867 %>% # Start with the 1867 Hansard data
  unnest_tokens(bigram, text, token = "ngrams", n = 2) # Create bigrams
```

Table 3.6: Raw bigrams extracted from the 1867 Hansard proceedings alongside their associated metadata.

sentence_id	speechdate	debate	year	bigram
S3V0185P0_0	1867-02-05	THE QUEEN'S SPEECH.	1867	being seated
S3V0185P0_0	1867-02-05	THE QUEEN'S SPEECH.	1867	seated on
S3V0185P0_0	1867-02-05	THE QUEEN'S SPEECH.	1867	on the
S3V0185P0_0	1867-02-05	THE QUEEN'S SPEECH.	1867	the throne
S3V0185P0_0	1867-02-05	THE QUEEN'S SPEECH.	1867	throne adorned

Note that the bigrams extracted overlap, as if the same sentence had been subdivided several times. The presentation of the bigrams is expected, as bigrams are created by looking at each pair of words in order, one step at a time—so each new bigram shares a word with the one before it.

Counting Bigrams

Now we can count the top bigrams and remove any that contain stop words. We remove stop words to focus on word pairs that might be more meaningful for our analysis.

The `filter(!str_detect(bigram, paste0(...)))` line removes rows where the bigram column contains a stop word. The `paste()` function combines the words in `stop_words$word` using the or operator (`|`), while `paste0()` adds word boundaries (`\\b`) around the regular expression so that only complete words are matched. The `str_detect()` function then identifies bigrams containing these stop words, and the `!` reverses the result so that only bigrams without stop words are retained.

`filter(str_detect(bigram, "^[a-z]+ [a-z]+$"))` keeps rows where the bigram column follows a specific pattern. The bigram must contain exactly two lowercase alphabetic words separated by a single space. Bigrams containing numbers, punctuation, or uppercase letters are excluded.

```
# Clean the bigrams in the 1867 Hansard dataset
# - str_detect() identifies and removes bigrams that contain common stop words
# - The second str_detect() keeps only bigrams made up of two lowercase alphabetic words
clean_bigrams_1867 <- bigrams_1867 %>%
  filter(!str_detect(bigram,
                     paste0("\\b(", paste(stop_words$word, collapse = "|"), ")\\b"))) %>%
  filter(str_detect(bigram, "^[a-z]+ [a-z]+$"))
```

Table 3.7: Cleaned bigrams from the 1867 Hansard corpus together with their metadata. The cleaned bigrams reflect preprocessing steps that remove stop words.

sentence_id	speechdate	debate	year	bigram
S3V0185P0_0	1867-02-05	THE QUEEN'S SPEECH.	1867	throne adorned
S3V0185P0_0	1867-02-05	THE QUEEN'S SPEECH.	1867	regal ornaments
S3V0185P0_1	1867-02-05	THE QUEEN'S SPEECH.	1867	robes sitting

sentence_id	speechdate	debate	year	bigram
S3V0185P0_2	1867-02-05	THE QUEEN'S SPEECH.	1867	robes commanded
S3V0185P0_2	1867-02-05	THE QUEEN'S SPEECH.	1867	gentleman usher

We can now count the resulting bigrams.

```
# Count the frequency of each bigram
# - count() counts the occurrences of each bigram
# - top_n() keeps the 100 most frequent bigrams
# - arrange() sorts the results from most to fewest occurrences
top_bigrams_1867 <- clean_bigrams_1867 %>%
  count(bigram) %>% # Count the occurrences of each bigram
  top_n(100) %>% # Keep the 100 most frequent bigrams
  arrange(desc(n)) # Sort from most to fewest occurrences
```

Table 3.8: The most frequent bigrams in the 1867 Hansard corpus, ranked by their number of occurrences.

bigram	n
noble lord	1807
noble earl	1538
noble friend	1027
roman catholic	943
reform bill	742

In our list, we see several contemporary rhetorical figures that stem from the practice of referring to other speakers in parliament as "my noble friend" or "my learned friend." We also see allusions to the debates over representation and the question of who would have the vote, especially in the phrases "household suffrage," referencing historical argument for granting middle-class men the right to vote with the Reform Bill of 1832, and "roman catholic," a reference to the fact that Roman Catholics could already vote until the Reform Act of 1834. Only in 1867 did those working-class men get the vote in Britain, and these earlier precedents were frequently alluded to as a justification.

We also see a reference to "public meetings." Why did speakers in parliament invoke public meetings in the lead-up to the Second Reform Act? We will answer this question by reading the original text.

```
# Keep speeches that mention the phrase "public meetings"
# - str_detect() searches the text for the specified phrase
my_reading <- hansard_1867 %>% # Start with the 1867 Hansard data
  filter(str_detect(text, "public meetings")) # Keep speeches containing the phrase

# Display sample speeches containing "public meetings"
# - head() keeps the first six rows
# - str_wrap() wraps long text for readability
# - select() keeps only the text column
```

```
# - kable() formats the results as a table
my_reading %>%
  head() %>% # Keep the first six rows
  mutate(text = str_wrap(text, 80)) %>% # Wrap long text
  select(text) %>% # Keep only the text column
  kable(caption = "Sample Hansard text from 1867 containing multiple occurrences
                   of the phrase \"public meetings.\"") # Format the results as a table
```

Table 3.9: Sample Hansard text from 1867 containing multiple occurrences of the phrase "public meetings."

text
I am aware that many of the public meetings have declared in favour of manhood

suffrage, but hardly any one in either House of Parliament wishes for such a suffrage, and if you give the working classes of this country a fair proportion of votes you may hope to settle this question for a considerable time. | |They are often drawn up with considerable ability; but they bear the mark, I think, of a single hand, and though they profess to emanate from public meetings in the different counties of Nova Scotia, they are— | |Speeches at public meetings, and the discussions of the press, great as their influence undoubtedly is, have yet less effect upon public opinion than the debates of Parliament. | |I know I shall be asked, from what I have seen in the press, and what has been said at public meetings, in respect to a plan that has been spoken of for workhouse and infirmary alteration, "Why do you not have six or seven large hospitals, each containing 1, 000 inmates, and conducted upon the principle of hospitals pure and simple?" | |Have you not heard or read what has been said at public meetings of every kind? | |There have been more than 1, 000 public meetings; at every one, the doors were open, and any man could come in, whatever his opinions, and the all but unanimous vote of these meetings has been in favour of something much larger and more complete than the plan of the right hon. |

A cursory reading of the sentences suggests that members of parliament were well aware of massive public meetings where more than a thousand members of the public showed up support the vote for working-class people. We read of demonstrations around British Empire, even in Nova Scotia. We read of members of parliament challenging each other to meet the demands of the public: "Have you not heard or read what has been said?" We have only skimmed a few sentences, but the material in these 126 rows gives us what we need to understand how members of parliament talked about public protests in these crucial months of 1867.

Bigrams, however, have limitations. A bigram only looks at one preceding word, but meaning often depends on words far apart in a sentence. Observing longer utterances—such as by moving beyond bigrams to multiword phrases—allows us to capture linguistic meaning that depends on more distant context than just the immediately preceding word.

Counting Multiword Phrases

Text mining also allows us to look for multiword phrases beyond bigrams. We can look at 5-word phrases from the year 1867. We can also add a line to filter the n-grams for the presence of the word "people," looking for voices that refer to democracy, for instance, "the will of the people," an important concept in the

years leading up to 1867, when working-class men in cities got the right to vote for the first time. The results will allow us to ask the question: What are the most frequently-invoked phrases involving "the people"?

We can use a slight adjustment of the code we used above to count bigrams to find n-grams of any length. Here is code for finding common five-word-phrases, filtering them, counting them, and finding the top 50 examples:

```
# Generate 5-grams (five-word sequences) from the text
# - unnest_tokens() tokenizes the text
# - token = "ngrams" creates sequences of words instead of single words
# - n = 5 creates five-word sequences (5-grams)
# the new column containing the 5-grams is called ngram
fivegrams_1867 <- hansard_1867 %>% # Start with the 1867 Hansard data
  unnest_tokens(ngram, text, token = "ngrams", n = 5) # Create 5-grams

# Identify the most frequent 5-grams that contain the word "people"
# - str_detect() keeps 5-grams that contain the word "people"
# - count() counts the frequency of each 5-gram
# - top_n() keeps the 50 most frequent 5-grams
people_n_grams <- fivegrams_1867 %>% # Create a new dataset
  filter(str_detect(ngram, "people")) %>% # Keep 5-grams containing "people"
  count(ngram) %>% # Count the frequency of each 5-gram
  top_n(50) # Keep the top 50 by count

head(people_n_grams)
```

```
## # A tibble: 6 x 2
##   ngram                           n
##   <chr>                       <int>
## 1 feelings of the people of       8
## 2 great body of the people       32
## 3 great majority of the people   13
## 4 great mass of the people       15
## 5 majority of the irish people   10
## 6 majority of the people of      27
```

We see a great many political concepts invoked about what members of parliament believed they were working for – the people of Ireland, England, or the metropolis, their representation, condition, education, minds, interests, feelings, rights, and welfare. An entire vocabulary of social engagement and political will was being spelled out.

Reading further down the list of multi-word phrases might give us more hints about the language with which speakers urged the principle of representation. Again, the careful analyst would trace the vocabulary words back to their original context, using the original text of the speeches to develop an argument about why a phrase like "the minds of the people" was used.

```
# Keep speeches that mention the phrase "minds of the people"
# - str_detect() searches the text for the specified phrase
```

```r
minds <- hansard_1867 %>% # Start with the 1867 Hansard data
  filter(str_detect(text, "minds of the people")) # Keep speeches containing the phrase

# Display sample speeches containing "minds of the people"
# - head() keeps the first six rows
# - str_wrap() wraps long text for readability
# - select() keeps only the text column
# - kable() formats the results as a table
minds %>%
  head() %>% # Keep the first six rows
  mutate(text = str_wrap(text, 80)) %>% # Wrap long text
  select(text) %>% # Keep only the text column
  kable(caption = "Sample Hansard text from 1867 containing multiple
                   occurrences of the phrase \"minds of the people.\"") # Format the results as a table
```

Table 3.10: Sample Hansard text from 1867 containing multiple occurrences of the phrase "minds of the people."

text
It is satisfactory to learn that these measures afforded great relief to the

minds of the people of Killarney, and that no further outbreak has occurred, except, as we understand, that the police barrack at Kells, eight miles from Cahirciveen, was attacked by the same party which visited Cahirciveen. | |Its debates have been the principal means by which political wisdom, and the results arrived at by the patient researches of philosophical inquirers, have been made gradually to sink into the minds of the people, and truths at first recognised only by a few of the ablest men of their day, have at length been practically adopted in legislation. | |Its debates command nothing like the same interest and attention, and exercise far less influence for good on the minds of the people. | |Gentleman the Question of which he had given notice, and was sure the Government would sacrifice nothing of its importance or dignity by acceding to that request; and if they could dispense with long debates on the abstract question, and on hypothetical grounds, and get to the investigation of the measure itself, it would tend not only to maintain the reputation of the Government, but what was still more important, the reputation and honour of the House, and the just influence which it ought to exercise over the minds of the people of this country. | |He believed, however, that the partial administration of justice in Ireland sometimes created an irritation in the minds of the people. | |The question is whether the impression is or is not to be conveyed to the minds of the people of this country that the House is in earnest upon this subject. |

When speakers referenced the "minds of the people," they frequently depicted popular politics as a form of collective political achievement to which ordinary people contributed the rational evidence of their own lived experience. We find the phrase connected to ideas about moral "duty" and political "harmony." We also see it invoked in reference to Ireland, India and Scotland.

These images of the peoples' collective wisdom contrast against the bias expressed by elites in another age, which warned against democracy as a form of potential despotism governed by ignorance. It is not surprising that the months leading into the Reform Act of 1867 would see eulogies to the wisdom of popular politics, but the phrase "the minds of the people" offers one entry-point for analysis.

Using Bigrams to Find Mentions of Events

For many kinds of textual analyses, incorporating external information can help guide the focus of the analysis and ensure it aligns with specific research questions. For example, a historian analyzing data might be interested in tracking references to notable events in British history, such as the Magna Carta, the Spanish Inquisition, or the Glorious Revolution, to see how often these events are mentioned in parliamentary records. To analyze how often these events are mentioned, the analyst may create and use a "controlled vocabulary" which is a predefined list of terms or topics of interest related to the analysis. `hansardr` includes an example controlled vocabulary: a curated list of significant events in British history, along with the corresponding years of their occurrence.

In applying our list, this exercise invites readers to investigate the idea that representations of memory change over time using the controlled vocabulary to search for a set group of phrases. We will use a controlled vocabulary to measure Parliamentarians' references to past events in the 1860 debates. We are intentionally choosing words such as "riot" or "meeting" that might collect evidence of parliamentary speakers referencing public meetings like those we saw evidence of above. We are also curious about how those references compare with references to famines, wars, strikes, exhibitions, and other contemporary and historical events.

```
# Load the Hansard speech data and debate metadata for the 1860s
data("hansard_1860")
data("debate_metadata_1860")

# Add debate metadata to the Hansard text
# - left_join() automatically detects that both datasets share the sentence_id column
# - it uses sentence_id to match each sentence with its corresponding metadata
hansard_1860 <- left_join(hansard_1860, debate_metadata_1860)

# Create a dataset containing speeches from 1867 before the Second Reform Act
# - mutate() extracts the year from the speech date
# - filter() keeps only speeches from 1867 through August 15
hansard_1867 <- hansard_1860 %>% # Start with the 1860s Hansard data
  mutate(year = year(speechdate)) %>% # Extract the year from the speech date
  filter(year == 1867, # Keep only speeches from 1867
         speechdate <= as.Date("1867-08-15")) # Keep speeches on or before August 15, 1867
```

```
# Create bigrams from the "text" column of the 1867 Hansard data
# - unnest_tokens() splits the text into sequences of consecutive words
# - token = "ngrams" specifies that the text should be tokenized into n-grams
# - n = 2 specifies that each n-gram should contain two consecutive words
# - The new column containing the bigrams is called "bigram"
bigrams_1867 <- hansard_1867 %>%
  unnest_tokens(bigram, text, token = "ngrams", n = 2)

# Remove unwanted bigrams
# - str_detect() identifies bigrams containing stop words
# - filter() removes bigrams containing stop words
# - the second filter() keeps only bigrams made of two lowercase words
clean_bigrams_1867 <- bigrams_1867 %>%
```

```
  filter(!str_detect(bigram, paste0("\\b(",
                                    paste(stop_words$word, collapse = "|"), ")\\b"))) %>% # Remove bigr
  filter(str_detect(bigram, "^[a-z]+ [a-z]+$")) # Keep only two-word lowercase bigrams

head(clean_bigrams_1867)
```

```
## # A tibble: 6 x 5
##   sentence_id speechdate debate               year bigram
##   <chr>       <date>     <chr>               <dbl> <chr>
## 1 S3V0185P0_0 1867-02-05 THE QUEEN'S SPEECH.  1867 throne adorned
## 2 S3V0185P0_0 1867-02-05 THE QUEEN'S SPEECH.  1867 regal ornaments
## 3 S3V0185P0_1 1867-02-05 THE QUEEN'S SPEECH.  1867 robes sitting
## 4 S3V0185P0_2 1867-02-05 THE QUEEN'S SPEECH.  1867 robes commanded
## 5 S3V0185P0_2 1867-02-05 THE QUEEN'S SPEECH.  1867 gentleman usher
## 6 S3V0185P0_2 1867-02-05 THE QUEEN'S SPEECH.  1867 black rod
```

The following code filters the `clean_bigrams_1867` dataset to retain only rows where the bigram column contains any word from the pattern1 list (e.g., "riot," "meeting," "famine"). It uses a regular expression to match whole words, ensuring accurate filtering. After filtering, the resulting dataset, `events_1867`, includes only the bigram and speechdate columns. Filtering allows the analyst to focus on specific bigrams related to the predefined topics of interest.

```
# Create a list of event-related keywords
# - this list will be used to search for bigrams about historical events
controlled_vocab = c("riot", "meeting", "famine", "revolt", "exhibition", "massacre",
                     "strike", "war")

# Keep bigrams containing one of the event keywords
# - paste() combines the keywords into a single search pattern
# - str_detect() searches each bigram for a keyword
# - select() keeps only the bigram and speech date
events_1867 <- clean_bigrams_1867 %>%
  filter(str_detect(bigram,
                    paste0("\\b(", paste(controlled_vocab, collapse = "|"), ")\\b"))) %>% # Keep bigram
  select(bigram, speechdate) # Keep only the bigram and speech date

head(events_1867)
```

```
## # A tibble: 6 x 2
##   bigram          speechdate
##   <chr>           <date>
## 1 sanguinary war  1867-02-05
## 2 dreadful famine 1867-02-05
## 3 late war        1867-02-05
## 4 civil war       1867-02-05
## 5 civil war       1867-02-05
## 6 china war       1867-02-08
```

If we look carefully at the events list so generated, we will recognize only a few of the references as actual events. Many of the bigrams including our controlled vocabulary are simply descriptions, for example, "disorderly riot." A few are references to actual events and meetings specified by the name of an event, for instance, the "Bristol Riot" or a "Yorkshire meeting." Far more frequent are general references to a "public meeting" or "recent meeting." All of the results are interesting, of course, because as we have seen members of parliament leaned on each other to acknowledge the public sentiment for expanding the vote.

How did references to meetings vary over time? By this time, it might be clear that searching for a controlled vocabulary, visualization, guided reading, and deliberation about the meaning of the data is an iterative process that we engage over many rounds – not an automatic process where the analyst looks for a word like "riot" and immediately finds meaningful results. We can search for our key phrases and visualize them.

In the following code, we define a new list of bigrams that make up our controlled vocabulary—specific phrases referring to historical events—and then filter the dataset to keep only the rows where the phrases appear. Note that the code below uses a new command for faceted counting–`group_by()`–which we will investigate in greater detail throughout the book.

For now, it is just important to know that `group_by()` effectively sorts data into buckets based on some category. For example, we could `use group_by()` to organize the data by year—putting all the speeches from the same year into one group—or by speaker, grouping together everything each person said. By grouping, we are able to perform calculations on each category individually. Instead of counting words for an entire data subset, like we did in Chapter 1, we can count words by group. In the following code we use `group_by()` to combine and count each bigram.

```
# Define a controlled vocabulary of specific multi-word historical events (as bigrams)
controlled_vocab_events = c("yorkshire meeting", "clontarf meeting", "recent meeting",
                            "public meeting", "paris exhibition", "orissa famine",
                            "irish famine", "crimean war", "civil war", "affghan war",
                            "kaffir war", "bristol riot")

# Identify the 10 most frequently mentioned historical events
# - filter() keeps bigrams that exactly match a defined historical event
# - group_by() groups occurrences of the same event
# - summarize() counts the total occurrences of each event
# - slice_max() keeps the 10 most frequently mentioned events
# - arrange() sorts the events from most to least frequent
top_events_1867 <- events_1867 %>%
  filter(bigram %in% controlled_vocab_events) %>% # Keep defined events
  group_by(bigram) %>% # Group by event
  summarize(total = n()) %>% # Count occurrences
  slice_max(total, n = 10) %>% # Keep top 10
  arrange(desc(total)) # Sort by frequency

head(top_events_1867)
```

```
## # A tibble: 6 x 2
##   bigram           total
##   <chr>            <int>
## 1 public meeting     103
```

```
## 2 crimean war         55
## 3 paris exhibition    35
## 4 civil war           31
## 5 recent meeting      11
## 6 orissa famine        8
```

```
# Add speech-date information to the most frequently mentioned events
# - left_join() adds speech dates from the full events dataset
# - mutate() extracts the month from each speech date
# - count() counts event mentions by speech date
# - ungroup() removes any remaining grouping
top_events_1867_w_speech_metadata <- top_events_1867 %>%
  left_join(events_1867, by = "bigram") %>% # Add speech dates
  mutate(month = month(speechdate)) %>% # Extract month
  count(bigram, speechdate) %>% # Count mentions by date
  ungroup() # Remove grouping

head(top_events_1867_w_speech_metadata)
```

```
## # A tibble: 6 x 3
##   bigram       speechdate     n
##   <chr>        <date>     <int>
## 1 affghan war  1867-02-26     2
## 2 affghan war  1867-07-26     1
## 3 bristol riot 1867-06-28     1
## 4 civil war    1867-02-05     5
## 5 civil war    1867-02-07     1
## 6 civil war    1867-02-14     1
```

In this visualization, it may be clear that we have produced a different sort of visualization than the bar charts we have been creating. `ggplot2` is a vast library for creating many different types of visualizations from the same dataset. Rather than the command for a bar chart – `geom_col()` – we have used a command for a dot plot – `geom_point()`.

```
# Create a bubble plot showing how often events were mentioned over time
# - aes() maps dates, event names, and frequencies to the plot
ggplot(top_events_1867_w_speech_metadata,
       aes(x = speechdate, # Plot the speech date
           y = bigram, # Plot the event bigram
           size = n, # Scale bubble size by the number of mentions
           color = n)) + # Scale bubble color by the number of mentions

  # - scale_color_viridis() formats the color scale
  scale_color_viridis(breaks = round, # Round the legend labels
                      trans = "log", # Log-transform the color scale
                      option = "A", # Use the Viridis A color palette
                      discrete = FALSE, # Treat n as continuous
```

```
                    direction = 1) + # Use the default color direction

  # - scale_x_date() formats the x-axis dates
  # - scale_size_continuous() scales the bubble sizes
  # - geom_point() draws the bubbles
  # - coord_cartesian() allows elements outside the plot region
  scale_x_date(date_breaks = "1 month", date_labels = "%B") + # Show one month per tick
  scale_size_continuous(range = c(1, 10)) + # Scale bubble sizes
  geom_point(alpha = .5, shape = 15) + # Draw semi-transparent square points
  coord_cartesian(clip = "off") + # Allow elements outside the plot region

  # - theme() customizes the plot appearance
  # - guides() hides the shape legend
  theme(legend.position = "bottom", # Place the legend below the plot
        axis.text.x = element_text(angle = 45), # Rotate x-axis labels
        plot.margin = unit(c(1, 20, 1, 1), "lines")) + # Increase the right margin
  guides(shape = "none") + # Hide the shape legend

  # - labs() adds axis labels and a subtitle
  # - ggtitle() adds the main title
  labs(x = "when parliament spoke", # Label the x-axis
       y = "events mentioned in the past by number of mentions", # Label the y-axis
       subtitle = "In the Months Leading Up to August 1867") + # Add a subtitle
  ggtitle("Events Mentioned by Name") # Add the main title
```

What do we make of this timeline? It suggests a swell of public meetings referenced in parliament in the months leading up to August 1867, which received more intense attention than either the Crimean War or the Paris Exhibition. Individual meetings such as those in Yorkshire or Clontarf received passing acknowledgment rather than deeper debate. Understanding the results of the visualization deeper requires more cycles of research and reading.

Other research questions appear that the individual analyst, engaged in a process of research, would want to follow. Did references to public meetings change from January to August, while their mentions intensified? Were all of these public meetings about the vote? What was the meaning of references to a "civil war" – the English civil war or recent events in America – and was this civil war invoked in reference to 1867? What about the references to the contemporary famine in Orissa (in India) or the (presumably historical) famine in Ireland?

One way to approach this line of inquiry would be by using the graph we have generated as a source of concrete questions about how members of parliament invoked current and historical events in the lead-up to the vote on the Second Reform Act. An analyst of history would follow up on most of these questions by reading individual sentences, speeches, and entire debates and to form these readings into a historical argument about the role of events in shaping political reform.

Finding Historical Events Using a Controlled Vocabulary

Next, let's return to the idea of finding events using what we have learned about joins. The authors of this book have created a controlled vocabulary of event names which lists the date of famous events like the

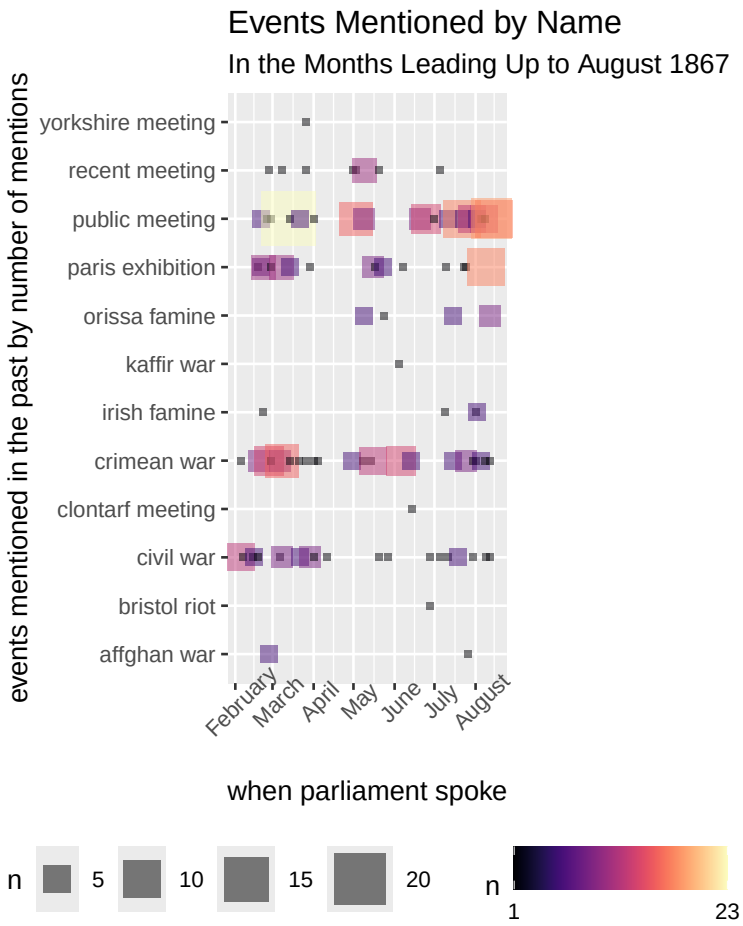


Figure 1: Plot showing the frequency with which Parliament in 1867 referred to major past events—including famines, wars, riots, and public meetings—across the months from February to August. Each square represents a day on which a speech mentioned a given event bigram; larger and brighter squares indicate higher numbers of mentions.

Magna Carta (1215).

As *The Dangerous Art of Text Mining* makes clear, we understand that there is no universal definition of the most important events in world history. Any controlled vocabulary list has its merits and its omissions. It is in comparing the subtleties of different lists of events that the historian begins to explore the changing political and cultural nature of memory – for example, the fact that references to the Glorious Revolution began to disappear after the First Reform Act, gradually displaced by allusions to the Tudor past.

```
# Load the scholar created "events" dataset from hansardr
data("events")

# Create a cleaned list of historical events and their assigned dates
# - distinct() removes duplicate event and date combinations
# - filter() excludes events dated after 1840
# - mutate() converts event names to lowercase for matching
# - select() keeps only the event name and assigned date
eventslist <- events %>% # Create a new dataset
  distinct(event_name, scholar_assigned_date) %>% # Remove duplicates
  filter(!scholar_assigned_date > 1840) %>% # Exclude post-1840 events
  mutate(event_name = tolower(event_name)) %>% # Lowercase event names
  select(event_name, scholar_assigned_date) # Keep name and date

head(eventslist)
```

```
## # A tibble: 6 x 2
##   event_name          scholar_assigned_date
##   <chr>                               <dbl>
## 1 french revolution                    1789
## 2 magna carta                          1215
## 3 norman conquest                      1066
## 4 corn laws                            1815
## 5 battle of boyne                      1690
## 6 glorious revolution                  1688
```

Notice that events contains two columns: events by their name, and the historical dates on which those events occurred.

We will now count events mentioned in parliamentary speeches, starting with using an `inner_join()`. Like its cousin, `left_join()`, `inner_join()` works with two data sets to make a new, combined data set. We use `inner_join()` when we want to analyze shared areas of overlap between two data sets. For our analysis. we want to find the n-grams from `annotated_1830` that are also listed as entities in the events controlled vocabulary.

```
# Load the Hansard speech text and metadata for the 1830s
data("hansard_1830")
data("debate_metadata_1830")

# Add debate metadata to the Hansard text
```

```
# - left_join() automatically matches rows using the shared sentence_id column
hansard_1830 <- hansard_1830 %>% # Start with the 1830 Hansard data
  left_join(debate_metadata_1830) # Add the debate metadata

# Generate bigrams (two-word sequences) from the text
# - unnest_tokens() tokenizes the text
# - token = "ngrams" creates sequences of words instead of single words
# - n = 2 creates two-word sequences (bigrams)
bigrams_1830 <- hansard_1830 %>% # Start with the 1830 Hansard data
  unnest_tokens(ngram, text, token = "ngrams", n = 2) # Create bigrams

# Keep only bigrams that match historical events
# - inner_join() matches bigrams to event names in eventslist
events_1830 <- bigrams_1830 %>% # Start with the bigram data
  inner_join(eventslist, by = c("ngram" = "event_name")) # Keep matching historical events

# Display sample historical event bigrams
# - head() keeps the first six rows
# - str_trunc() shortens long text for readability
# - kable() formats the results as a table
events_1830 %>%
  head() %>% # Keep the first six rows
  mutate(ngram = str_trunc(ngram, 120)) %>% # Shorten long event names
  kable(caption = "Sample two-word phrase events from the 1830 Hansard debates.") # Format the results
```

Table 3.11: Sample two-word phrase events from the 1830 Hansard debates.

sentence_id	speechdate	debate	ngram	scholar_assigned_date
S2V0022P0_158	1830-02-04	ADDRESS ON THE LORDS COMMISSIONERS SPEECH.]	navigation laws	1660
S2V0022P0_1722	1830-02-18	POUTUGAL.]	portuguese constitution	1822
S2V0022P0_1728	1830-02-18	POUTUGAL.]	portuguese constitution	1822
S2V0022P0_1735	1830-02-18	POUTUGAL.]	portuguese constitution	1822
S2V0022P0_1838	1830-02-18	POUTUGAL.]	portuguese constitution	1822
S2V0022P0_1928	1830-02-18	POUTUGAL.]	portuguese constitution	1822

Notice that the resulting dataset contains speaker and date columns from the parliamentary speeches, but the only bigrams listed are those that correspond to events from `eventslist`.

In the next blocks of code, we use our new list of events to generate a visualization.

```r
# Count event mentions by speech date
# - mutate() extracts the year from each speech date
# - group_by() groups mentions by event, year, speech date, and a scholar assigned date
# - summarize() counts the number of mentions in each group
# - ungroup() removes the grouping when no longer needed
counted_events <- events_1830 %>% # Create a new dataset
  mutate(year = year(speechdate)) %>% # Extract year
  group_by(ngram, year, speechdate, scholar_assigned_date) %>% # Group data
  summarize(n = n()) %>% # Count mentions
  ungroup() # Remove grouping

head(counted_events)
```

```
## # A tibble: 6 x 5
##   ngram                 year speechdate scholar_assigned_date     n
##   <chr>                <dbl> <date>                      <dbl> <int>
## 1 american constitution  1831 1831-10-05                   1776     1
## 2 american constitution  1831 1831-10-06                   1776     2
## 3 american constitution  1832 1832-04-13                   1776     2
## 4 belgian revolution     1831 1831-08-09                   1830     1
## 5 belgian revolution     1831 1831-08-18                   1830     1
## 6 belgian revolution     1832 1832-03-16                   1830    10
```

```r
# Summarize total mentions of each event and sort by assigned date
# - group_by() groups mentions by event and assigned date
# - summarize() calculates the total number of mentions for each event
# - ungroup() removes the grouping after the totals are calculated
# - arrange() sorts the events chronologically by their assigned date
cleaned_events <- counted_events %>%
  group_by(ngram, scholar_assigned_date) %>% # Group by event and date
  summarize(total = sum(n)) %>% # Calculate total mentions
  ungroup() %>% # Remove grouping
  arrange(scholar_assigned_date) # Sort chronologically

head(cleaned_events)
```

```
## # A tibble: 6 x 3
##   ngram              scholar_assigned_date total
##   <chr>                              <dbl> <int>
## 1 norman conquest                     1066     5
## 2 twelfth century                     1100     3
## 3 thirteenth century                  1200     7
## 4 fourteenth century                  1300     2
## 5 game laws                           1389   112
## 6 fifteenth century                   1400     7
```

```r
# Divide the historical date range into 30 time periods and identify the top event in each
# - mutate() assigns each event to one of 30 chronological time periods
# - group_by() groups events by time period
# - filter() keeps the most frequently mentioned event in each time period
# - ungroup() removes the grouping
# - left_join() adds the corresponding speech information
indexed_events <- cleaned_events %>%
  mutate(time_period = scholar_assigned_date %/%
           ((max(scholar_assigned_date) -
               min(scholar_assigned_date)) / 30)) %>% # Assign time period
  group_by(time_period) %>% # Group by time period
  filter(total == max(total)) %>% # Keep top event
  ungroup() %>% # Remove grouping
  left_join(counted_events,
            by = c("ngram", "scholar_assigned_date")) # Add speech data

head(indexed_events)
```

```
## # A tibble: 6 x 7
##   ngram           scholar_assigned_date total time_period  year speechdate     n
##   <chr>                           <dbl> <int>       <dbl> <dbl> <date>     <int>
## 1 norman conquest                  1066     5          41  1830 1830-11-15     1
## 2 norman conquest                  1066     5          41  1834 1834-03-14     1
## 3 norman conquest                  1066     5          41  1834 1834-04-25     1
## 4 norman conquest                  1066     5          41  1839 1839-07-12     2
## 5 twelfth century                  1100     3          43  1831 1831-07-06     1
## 6 twelfth century                  1100     3          43  1832 1832-02-27     1
```

```r
# Set the end date 8 days after the latest speech date
# - Adding 8 days provides extra space at the edge of the visualization
left_range <- max(events_1830$speechdate) + 8
```

```r
# Create a bubble plot showing when historical events were mentioned in Parliament
# - ggplot() creates the visualization using the indexed_events dataset
# - x = speechdate places the date of each mention on the x-axis
# - y = scholar_assigned_date places the historical date of each event on the y-axis
# - size = n changes the size of each point based on the number of mentions
# - label = paste0() creates a label with the event name and historical date
# - color = n changes the color of each point based on the number of mentions
ggplot(data = indexed_events, # Use the event data
       aes(x = speechdate, # Set the x-axis
           y = scholar_assigned_date, # Set the y-axis
           size = n, # Set point size
           label = paste0("           ", ngram, " (", scholar_assigned_date, ")"), # Create labels
           color = n)) + # Set point color

# Set the color scale for the bubbles
```

```r
# - scale_color_viridis() applies a color gradient based on the number of mentions
# - breaks = round rounds the values shown in the legend
# - trans = "log" uses a logarithmic scale to make differences easier to see
# - option = "A" selects the viridis color palette
# - discrete = FALSE treats the number of mentions as a continuous value
# - direction = 1 applies the color gradient from low to high values
scale_color_viridis(breaks = round, # Round legend values
                    trans = "log", # Use log scale
                    option = "A", # Set color palette
                    discrete = FALSE, # Use continuous scale
                    direction = 1) + # Set gradient direction

  # Set the appearance and placement of the bubbles
  # - scale_size_continuous() sets the range of bubble sizes
  # - geom_point() displays the data as semi-transparent square points
  # - coord_cartesian() allows labels to extend beyond the plot area
  # - scale_x_date() displays the x-axis in one-year intervals
  scale_size_continuous(range = c(1, 10)) + # Set bubble sizes
  geom_point(alpha = .5, shape = 15) + # Create square bubbles
  coord_cartesian(clip = "off") + # Allow labels outside plot
  scale_x_date(date_breaks = "1 year", date_labels = "%Y") + # Show yearly ticks

  # Add one label for each historical event
  # - group_by() groups the data by event
  # - sample_n() selects one mention of each event for labeling
  # - x = left_range places the labels at the right side of the plot
  # - y = scholar_assigned_date aligns each label with its historical date
  # - hjust = 0 left-aligns the label text
  geom_text(data = indexed_events %>% # Use event data
              group_by(ngram) %>% # Group by event
              sample_n(1), # Select one mention
              show.legend = FALSE, # Hide label legend
              aes(color = 1, # Set label color
                  x = left_range, # Position labels
                  y = scholar_assigned_date, # Align by event date
                  hjust = 0, # Left-align labels
                  size = 8)) + # Set label size

  # Adjust the plot layout for readability
  # - theme() moves the legend, adds space for labels, and adjusts the x-axis text
  # - guides() removes the shape legend
  theme(legend.position = "bottom", # Move legend below plot
        plot.margin = unit(c(1, 250, 50, 1), unit = "pt"), # Add space for labels
        axis.text.x = element_text(angle = 45, vjust = 0.5, hjust = 1), # Rotate year labels
        axis.title.x = element_text(vjust = -0.5)) + # Adjust title spacing
  guides(shape = "none") + # Remove shape legend
```

```
  # - labs() adds axis labels and a subtitle
  # - ggtitle() adds the main title
  labs(x = "when parliament spoke", # Assign the x-axis
       y = "events mentioned in the past") + # Assign the y-axis
  ggtitle("Events Mentioned by Name in Parliament") # Create a title
```

Note that the visualization here resembles the visualization of events we generated above, but in this case, we have some extra information: the actual date of the event mentioned, e.g. 1789 for the French Revolution. We have plotted this historical information on the y-axis. The resulting graph in fact shows two timelines rather than one – a timeline of parliamentary speech on the x-axis and a timeline of historical references on the y-axis.

How should we interpret this graph? One approach is to identify the most surprising aspects of the visualization and explore their implications. For instance, as a British historian, I am unsurprised by frequent references in Parliament to events from the previous half-century, such as the French and American Revolutions. Similarly, it's expected that the Glorious Revolution was heavily referenced in the early 1830s, with mentions tapering off after 1836, and that the Riot Act was invoked annually. However, the most striking feature of the graph may be the vertical line around 1832, where references to much older historical periods—the sixteenth, thirteenth, and twelfth centuries—appear. This invocation of the distant past invites further investigation into its context and significance.

Why? we find ourselves asking. That verticality of temporal experience–the long perspective on the deep past–seems to be fairly distinct for 1830-32 of all the years in the decade. Why do so many different, varied periods suddenly become relevant all at once? Is this pattern a general characteristic of moments of political reform – that the deep past is trawled for a multitude of examples, for and against?

We continue the explanation through reading.

```
# Extract the year from the speech date
# - mutate() creates a new year column from speechdate
hansard_1830 <- hansard_1830 %>%
  mutate(year = year(speechdate))

# Find speeches that mention the fifteenth century in 1832
# - filter() keeps only speeches from 1832
# - regex() searches for several forms of "fifteenth century"
# - ignore_case = TRUE matches uppercase and lowercase text
# - select() keeps only the text column
c15_mentions <- hansard_1830 %>% # Start with the 1830 Hansard data
  filter(year == 1832, # Keep only speeches from 1832
         str_detect(text,
                    regex("fifteenth century|fifteenth-century|15th century|15th-century",
                          ignore_case = TRUE))) %>% # Keep speeches mentioning the fifteenth century
  select(text) # Keep only the text column
```

```
# Display sample speeches mentioning the fifteenth century
# - head() keeps the first six rows
# - str_wrap() wraps long text for readability
```

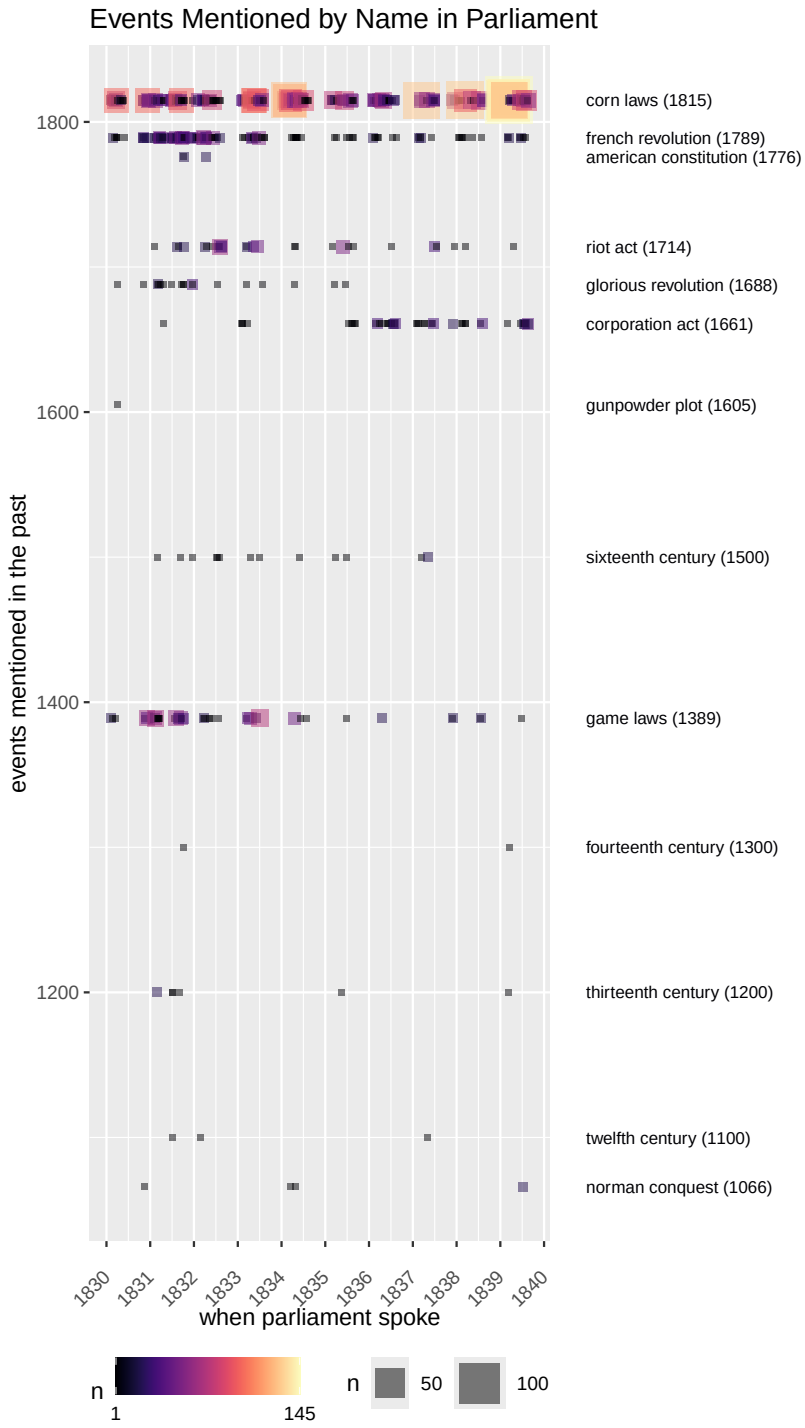


Figure 2: Plot of how often nineteenth-century MPs invoked earlier historical events in the debates of the 1830s. The x-axis shows when speeches were delivered (1830–1840), while the y-axis shows the historical date of the event being named. Each square marks at least one speech mentioning that event; larger and brighter squares indicate more frequent references, highlighting especially intensive discussion of late eighteenth- and early nineteenth-century events such as the Corn Laws, the French Revolution, and the

```
# - select() keeps only the text column
# - kable() formats the results as a table
c15_mentions %>%
  head() %>% # Keep the first six rows
  mutate(text = str_wrap(text, 80)) %>% # Wrap long text
  select(text) %>% # Keep only the text column
  kable() # Format the results as a table
```

text
The celebrated Act of William (1696) was only a transcript from a Scottish

law of the fifteenth century, which was itself the enforcement only, by civil sanction, of the original canon of the Catholic Church. |

```
# Filter speeches from 1832 that mention "sixteenth century"
# - filter() keeps speeches for just the year 1832
# - str_detect() keeps speeches containing the phrase "sixteenth century"
# - select() keeps only the text column
c16_mentions <- hansard_1830 %>% # Create a new dataset
  filter(year == 1832, # Filter for 1832
         str_detect(text,
                    regex("sixteenth century|sixteenth-century|16th century|16th-century",
                          ignore_case = TRUE))) %>% # Find references to the sixteenth century
  select(text) # Keep the text column
```

```
# Display sample speeches mentioning the sixteenth century
# - head() keeps the first six rows
# - str_wrap() wraps long text for readability
# - select() keeps only the text column
# - kable() formats the results as a table
c16_mentions %>%
  head() %>% # Keep the first six rows
  mutate(text = str_wrap(text, 80)) %>% # Wrap long text
  select(text) %>% # Keep only the text column
  kable() # Format the results as a table
```

text
In the middle of the sixteenth century, a law, which actually existed even at

present, was placed in the Statute-book, though it had fallen into desuetude, by which, whoever, not being a nobleman, should shoot wild-fowl, should suffer death, and forfeit all his goods and chattels, which were to be given to whoever should sue for the same. | |said, that the Court of Delegates, to which this Bill referred, had certainly been constituted originally by the issue of a mere Commission from the King, under which

their juridical and appellate judicial functions were exercised; but it had existed for so long a period, ever since the middle of the sixteenth century, in fact, that the abolition of a Court which had existed for so long a time, demanded somewhat more formality than the mere cessation of the issue of a Commission from the Crown, and for that reason it had been deemed proper that an Act of the Legislature should sanction that act of abolition. | |Every one knew that, in the sixteenth century, religion was the politics of the period. |

```
# Find speeches that mention the seventeenth century in 1832
# - filter() keeps only speeches from 1832
# - regex() searches for several forms of "seventeenth century"
# - ignore_case = TRUE matches uppercase and lowercase text
# - select() keeps only the text column
c17_mentions <- hansard_1830 %>% # Start with the 1830 Hansard data
  filter(year == 1832, # Keep only speeches from 1832
         str_detect(text,
                    regex("seventeenth century|seventeenth-century|17th century|17th-century",
                          ignore_case = TRUE))) %>% # Keep speeches mentioning the seventeenth century
  select(text) # Keep only the text column
```

```
# Display sample speeches mentioning the seventeenth century
# - head() keeps the first six rows
# - str_wrap() wraps long text for readability
# - select() keeps only the text column
# - kable() formats the results as a table
c17_mentions %>%
  head() %>% # Keep the first six rows
  mutate(text = str_wrap(text, 80)) %>% # Wrap long text
  select(text) %>% # Keep only the text column
  kable() # Format the results as a table
```

text
Not to keep you in suspense as to precedents, your Lordships must recollect that

remarkable instance to-wards the close of the 17th century, when Parliament, in a situation not dissimilar from the present, attempted, in the reign of Charles 2nd, to detach that Monarch from his unnatural connexion with France, and recommended to him in lieu thereof to form an alliance more creditable to his character and to the honour and interest of the country, by cultivating a closer and more cordial intimacy with the Dutch nation. | |He thought that Europe, in the nineteenth century was not less indebted to them than it was in the seventeenth century, for the heroic valour with which they had struggled against the common enemy of all its free states and liberal institutions. |

What we see is a collection of historical references which serve two purposes: one is explaining why the institutions of the past differ from the needs of the present, and one is invoking the fact that parliamentary institutions have changed in the past to support the cause of reform in the present. The arguments work together, not against each other.

While we acknowledge that other historians have studied the discourse of parliamentary reform in detail, and even arrived at this same conclusion, what we find from a data-driven analysis of historical references to past

events in parliament is the distinctiveness of 1832 as a moment of memory. Understanding how memory was used requires deep reading and a knowledge of the context of the debates over the vote; but understanding that historical memory was being leveraged in a particularly intensive way in 1832 is an insight that comes from treating text as data.

Data Processing Order

Throughout this chapter, we worked with large datasets and showed how to handle them effectively The process we demonstrated is especially important when handling a dataste as large the Hansard corpus. For example, if we try to tokenize the entire dataset before narrowing it down, the tokenization steps can exceed our computer's resources. Instead, by first filtering the dataset to include only the debates we care about–and then tokenizing–we can significantly reduce the processing load.

Here is a typical workflow for handling large data like Hansard:

- Load the Dataset: Import the dataset into the R environment using appropriate functions like `data()`.
- Filter or Subset the Dataset: Narrow down the dataset to include only the data relevant to the analysis. For example, if we are analyzing speeches by William Gladstone, we can filter rows where the speaker is Gladstone using functions like `filter()` from the `dplyr` package. Similarly, we could filter for speeches belonging to a specific debate by filtering the data based on a title. Subsetting early in the process reduces the amount of data that will be processed later, saving processing time and computational resources.
- Clean the Dataset: After subsetting, clean the dataset to prepare it for analysis. Cleaning typically involves handling missing values, removing stop words, and normalizing text (e.g., converting text to lowercase). We can clean data with functions like `mutate()` from `dplyr`, or string processing functions like `str_to_lower()`, which transforms text to lower case, from the `stringr` package.
- Analyze the Dataset: Now that the data has been filtered, apply analytical metrics to identify patterns and answer the research question. This may include counting occurrences, calculating averages or proportions, or using statistical methods to examine trends and relationships in the data.

Although it is possible clean the entire dataset before narrowing it down, doing so can strain our computer's processing power or even exceed its available memory. It is often more desirable to filter the dataset first, then clean the smaller, more manageable portion. Following the order: Load -> Subset -> Clean -> Analyze can optimize data processing, which may be especially useful when analyzing change over decades or centuries of data.

Exercises

1) In the code above, we searched for 5-word phrases that reference "the people." Use the code to find two-word phrases in 1830, 1840, 1850, and 1860 that reference "meeting." Write a paragraph, quoting at least 10 phrases from each decade. For each of the phrases mentioned more than five times, describe what prejudices, positive or negative, the phrase encapsulates. Use evidence to answer the question: how did parliamentary references to meetings change between 1830 and 1860?

Tips: Instead of joining every decade together, process the decades separately to avoiding using all the computer's resources.

2) In the code above, we created two kinds of timelines, one for 1867 using a controlled vocabulary of words such as "meeting" and the other for the 1830s using the list of historical events, `eventlist`. Make two more timelines. Apply the controlled vocabulary to the 1830s, and search for the items in `eventlist` in the 1860s. Use the code above to graph the results. Keep in mind that one may need to adjust the controlled vocabulary to what is found in the data.

3) Analyze the results of the timelines. Use iterative investigations of phrases, as modeled above, to conduct a distant reading of the parliamentary speeches. Using at least ten examples, compose a one-page essay investigating the question: how did parliamentary speakers use references to contemporary and historical events to make arguments for or against political change?

4) Compare the language of "reform" across decades by finding three-word phrases containing the word "reform." Count the 25 most common phrases in each decade. Then compare the results. Based on the top 25 words, does the language seem to have changed much between these decades? If not, can we find which phrases appear only in one decade and not the other using `anti_join()`?

5. How did Parliament describe "the people" in the 1830s? Generate five-word phrases containing "people" separately for each decade. Count the most frequent phrases.

6. How did Parliament refer to different forms of political protest in the 1830s and 1860s? Use `str_detect()` and `regex()` to search the original speeches for multiple forms of protest-related words, such as "protest", "protests", "protesting", "agitation", "agitate", "demonstration", and "demonstrations". Compare the results between the two decades. Read at least ten examples from each decade and answer: Did parliamentary descriptions of political protest change between the 1830s and 1860s?

7. Can distant reading help us arrive at questions that close reading can answer? Create one of the chapter's event timelines for a new period. Identify one surprising cluster, increase, absence, or unusual historical reference in the visualization. Then use `str_detect()` to return to the relevant speeches and explain what produced the pattern.

8. Do the most frequent phrases necessarily provide the most historically useful evidence? Generate the top 100 bigrams for one year or period and compare the highest-frequency phrases with less frequent phrases connected to the chapter's controlled vocabulary. Which phrases lead to more useful understanding of Hansard, and why?